Computers and Electrical Engineering 110 (2023) 108852

Contents lists available at ScienceDirect

Computers and Electrical Engineering

journal homepage: www.elsevier.com/locate/compeleceng

An intelligent network monitoring approach for online classification of Darknet traffic ☆

Rodrigo Moreira [a,*], Larissa Ferreira Rodrigues Moreira [a,b], Flávio de Oliveira Silva [b]

[a] Institute of Exact and Technological Sciences (IEP), Federal University of Viçosa (UFV), Rio Paranaíba, 38.810-000, Minas Gerais, Brazil
[b] Faculty of Computing (FACOM), Federal University of Uberlândia (UFU), Uberlândia, 38.400-902, Minas Gerais, Brazil

A R T I C L E I N F O

Keywords:
Darknet
Deep learning
Network sensing
Adaptive sampling
Reinforcement learning
Monitoring

A B S T R A C T

The Internet plays a crucial role in supporting global applications and businesses, but security remains a major challenge. Within the Internet, there exists a parallel network known as the Darknet, where malicious activities and traffic are present and require real-time classification. Many methods aim to classify this Darknet traffic in real-time due to its significant volume within Internet traffic. However, online Darknet traffic classification faces challenges, particularly in determining the optimal packet sampling amount for achieving a high classification rate in high-performance networks. To address this, our paper presents a novel approach that combines Convolutional Neural Network (CNN) and Reinforcement Learning (RL) techniques for intelligent and adaptive packet sampling rates in high-performance network interfaces. This method reduces overhead on monitored entities, especially in high-speed networks with a high bit rate. Our findings demonstrate a TOR traffic prediction accuracy of 99.84% and successful classification tasks in high-throughput networks using our method.

1. Introduction

Infrastructure monitoring is essential in order to understand the resource behavior, memory consumption, and availability and support quality of the applications that run on them [1]. The Internet consists of heterogeneous types of infrastructure, with numerous entities, users, and services that support its operation and maintain connectivity for essential applications such as those used in finances, health, and business [2]. The infrastructure of the Internet enables the hosting and reachability of pages, known as the Surface Web, indexed by conventional search engines and make up approximately 4% of the Internet traffic.

On top of the same infrastructure runs an overlay network called the Darknet, which was developed in 1971 at the Massachusetts Institute of Technology (MIT) [3]. Its pages are not indexed by conventional search engines, as shown in Fig. 1. The Darknet forms part of the Deep Web, which has more than 500 times more traffic than the Surface Web [4]. On top of this overlay network, there is traffic consisting of illegal activities such as hacking, phishing, and other crime, and frauds which is available through software, specific protocols, and configurations [5].

The Onion Router (TOR) is the standard method of accessing the content of this overlay network, which mainly carries encrypted traffic [6]. Classifying this type of traffic is a relevant topic, and requires investigation both the industrial purposes and to allow governments to carry out national crackdowns and supervision [7]. Artificial Intelligence (AI) techniques have been successfully used or have been combined with other methods of classifying and predicting this type of traffic over network infrastructures [5]. These The aim of these efforts has been to classify TOR and *non*-TOR traffic in order to predict suspicious activity [8]. In the following,

☆ This paper is for special section VSI-webc. Reviews were processed by Guest Editor Dr. M. Manikandan and recommended for publication.

* Corresponding author.
E-mail addresses: rodrigo@ufv.br (R. Moreira), larissarodrigues@ufu.br (L.F. Rodrigues Moreira), flavio@ufu.br (F. de Oliveira Silva).

https://doi.org/10.1016/j.compeleceng.2023.108852
Received 14 July 2022; Received in revised form 28 June 2023; Accepted 30 June 2023
Available online 14 July 2023

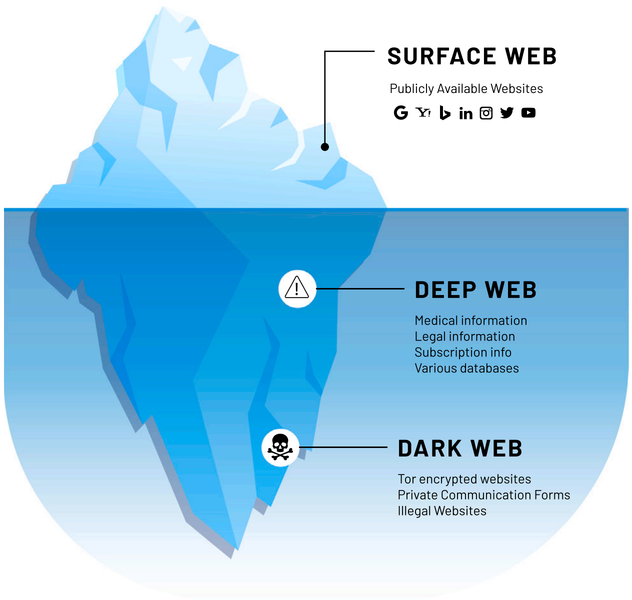


Fig. 1. Composition of internet traffic inspired by Singh et al. [3].

we refer to TOR traffic as Darknet (malicious) traffic, and to *non*-TOR activity as non-Darknet (benign). The classification of these types of real-time traffic with high precision still poses challenges.

Convolutional Neural Network (CNNs) are currently achieving good results in the areas of Natural Language Processing (NLP), image and video processing, and medical applications [9]. CNNs also seem to be suitable for network traffic classification tasks, due to their ability to learn and extract features, and their capacity to capture the spatial dependence between the bytes of a network packet [10,11]. The RL technique supports the detection of abnormal behavior on a network, and combining this approach with CNNs shows promise in terms of achieving more consistent results [12]. However, classifying Darknet and non-Darknet traffic with high accuracy requires large amounts of computational resources [3], giving rise to a need for low-power techniques using new packet sampling techniques for traffic rating [13].

From examining the methods in the literature, we discovered that many authors have used Artificial intelligence (AI) techniques to handle traffic classification problems [14,15], and particularly CNNs [3,16]. In addition, many efforts have focused on predicting Darknet and non-Darknet traffic [17,18]. We believe this paper is the first to propose advances in the online classification of this type of traffic while advancing the state of the art with an intelligent packet sampling technique.

The strategies in the literature for classifying TOR and non-TOR traffic show strong potential, but these mainly focus on classifying traffic as malignant or benign within a set of known application classes. Methods that are capable of handling this classification problem in real time (or close to it) are challenging and still need breakdowns. One important challenge involves determining the optimal number of packets to sample on a network in order to maintain a high rate of detection accuracy while minimizing the amount of computational resources needed to process a burst of packets. We hypothesize that an approach that combines CNNs with RL will be suitable for near-real-time traffic profile classification. In our method, the CNNs are used to deal with traffic classification, while RL is used to find the optimal amount of packets to sample at each instant to reach the maximum possible traffic profile detection rate.

To address this gap in state-of-the-art research, this paper proposes a method for the online classification of Darknet and *non*-Darknet traffic. Our technique is novel in that it combines technologies such as CNNs with RL for online traffic prediction. We combined these two methods to achieve suitable rates of online sampling and classification in high-performance networks, with the aim of the avoiding the overloading of the monitoring entities through an intelligent sampling approach.

The main contributions of this paper include the following:

- We propose and evaluate of a high-accuracy classifier for TOR and *non*-TOR traffic prediction.
- We present an adaptive packet sampling method for traffic classification.
- We propose and evaluate a method for the online classification of packets based on CNNs.
- We present an evaluation of and inferences related to the role of processing time in online traffic classification.
- We compare our method with other state-of-the-art approaches for Darknet traffic classification.

The remainder of the paper is organized as follows: Section 2 gives an overview of related works and identifies their main differences from the present study. Section 3 introduces the proposed method we used for online classification of TOR and *non*-TOR traffic, while Section 4 presents the experimental testbed and the dataset used. In Section 5, we discuss and highlight the main findings of this work, and in Section 6, we present a summary of the paper.

2. Related work

Classifying network traffic is essential for operators to know how resources are consumed and maintain the Service Level Agreement (SLA) with users. Therefore, some surveys dived into understanding how the network traffic classification and AI evolved monitoring technologies. In the literature, traffic classification approaches differ as feature-based or payload-based [19]. This section describes how the literature has addressed artificial intelligence to support the monitoring and classification of network traffic more effectively.

Dias et al. [14] proposed an approach available for classifying video traffic in real-time through its packets. In the application domain, both proposals are different, but they aim to classify real-time traffic. Therefore, it is imperative to highlight that the method of authors storing and pre-processing the packets for future classification. In addition, their approach predicts a ranking of variables that most influenced the prediction process. Differently, our method does not require storage or pre-processing packets in the online sorting process.

Aceto et al. [20] described and evaluated a CNN-based approach for mobile device traffic classification. Their approach bifurcates the dataset training flow into two distinct CNNs, merging the measured weights at the end into a single network. Unlike the authors, our proposal classifies Darknet and non-Darknet traffic in real-time based on a single CNN.

Zheng et al. [21] describe and evaluate an encrypted traffic classification method based on the raw information of the flows established between two end-to-end entities. The authors' method consists of artificially increasing the dataset using the *hallucinator* technique, training several CNNs simultaneously. Also, their method shares according to the Mean Squared Error (MSE) criterion, their parameters mutually. They carried out a comparison in terms of performance with some state-of-the-art techniques. Unlike the authors, our method classifies both Darknet and non-Darknet traffic with an RL-based technique for sampling traffic for real-time classification.

Lashkari et al. [16] proposed and evaluated a method based on feature extraction to detect and characterize Darknet and non-Darknet traffic. The authors' technique consists of creating an image based on extracting features collected by the CICFlowMeter tool. The images that feed the CNNs were generated from the feature vector that the CICFlowMeter generated after feature ranking. Unlike the authors' approach, we classify packets not as a flow but rather consider the entire package structure. Furthermore, our method can classify real-time traffic through an adaptive packet sampling mechanism designed for high-performance networks.

There is a network monitoring approach based on machine learning capable of automatically setting network thresholds and baselines under different conditions in Mijumbi et al. [22]. In their method, the collected data of the network elements are stored and fed to the Long Short-Term Memory (LSTM) for future predictions that are further improved through the network operators' feedback. Unlike the authors, our approach improves monitoring by using a reinforcement learning algorithm that determines how many network packets need to be sampled for a Darknet traffic prediction goal to be achieved.

Ujjan et al. [23] describe and evaluate a CNN-based pooling sampling approach to detect denial of service attacks in Software-defined Networking (SN). Similar to our method, the authors argue that the pooling intervals in a monitored network need to be adaptive since the flow characteristic can change suddenly depending on the network conditions. Network sampling with fixed parameters can impose overhead. In this sense, the authors predict the frequency of pooling based on historical data. Our proposal goes further and uses an RL algorithm for packet sampling intelligently to feed the traffic classifier with the best packet sample.

Iliadis and Kaifas [17] applied conventional machine learning algorithms and evaluated its performance in the task of classifying Darknet and non-Darknet traffic and in specific applications within each class. The evaluated techniques were K-Nearest Neighbors (KNN), Multilayer Perceptron (MLP), Gradient Boost (GB), Random Forest (RF), and Decision Tree (DT) for Darknet traffic classification and reached an accuracy of around 98%. Unlike the authors, ours goes further with superior accuracy and provides an online classification of Darknet and non-Darknet traffic.

Jadav et al. [18] proposed and evaluated a method based on dimensionality reduction and balancing for the classification of Darknet and non-Darknet traffic. Having reduced the dimensionality by the Synthetic Minority Oversampling Technique (SMOTE), which consists of synthetically increasing classes that are the minority, there was a positive impact on the classifiers' performance metrics. Unlike the authors, our proposal does not require prior dataset processing since we classify Darknet and non-Darknet traffic in real-time.

Marim et al. [15] proposed an offline classification method for Darknet and non-Darknet traffic using conventional classifiers such as DT and RF. Unlike the present article, the technique proposed by the authors does not allow online packet classification. Thus, the present work allows the adaptive sampling of packets for online prediction of the traffic class.

Singh et al. [3] used pre-trained CNNs for bi-level classification of Darknet and non-Darknet traffic. The authors' technique consists of building images from a matrix of features that are collected from flows. In contrast, our approach intelligently samples network packets for online classification of Darknet and non-Darknet traffic.

We summarize the primary efforts in the literature in classifying Internet traffic in Table 1. "Classification Goal" in Table 1 refers to the work target containing the objective of traffic classification. The Column "Method" directly references the ML technique used in the prediction or classification of traffic, and many works used classifiers with supervised learning [18].

The "Input" column describes the type of data that fed the ML engine for predicting/categorizing network traffic. In our short survey, we noticed that solutions are predominantly based on flows or statistics derived from this flows [17,20]. The "Output" column refers to the ML technique's output; most rely on Darknet traffic classification [3,24]. The column "Real-Time Classification" refers to the solution's ability to classify Darknet and non-Darknet traffic online. We choose the binary marker checked (✓) and not checked (×) to demonstrate whether the state-of-the-art solution has the feature.

Predominantly, state-of-the-art solutions still need to address real-time classification challenges consistently. Additionally, the "Intelligent Sampling" column refers to solutions' ability to adjust packet sampling size in a real-time way to feed our classification method. None of the surveyed state-of-the-art methods realized this intelligent packet sampling.

Table 1
Short state-of-the-art survey.

Approach	Classification goal	Method	Input	Output	Dataset	Real-time classification	Intelligent sampling
Dias et al. [14]	Streaming traffic	New algorithm based on Naive Bayes	Statistics Tuple	Netflix Youtube File Downloads	Generated by themselves	✓	×
Aceto et al. [20]	Mobile Application traffic	CNN	Flow Statistics Tuple	Mobile App Classes	FB-FBM Global Mobile Solutions	×	×
Zheng et al [21]	Encrypted traffic	Hallucinator AutoEnconder AutoDecoder	Flow Statistics Tuple	Internet common applications	ISCX 2012 IDS ISCXVPN2016	×	×
Lashkari et al. [16]	Darknet traffic	CNN	Image	Darknet or non-Darknet	ISCXVPN2016 ISCXTor2017	×	×
Mijumbi et al. [22]	Estimates best network thresholds for monitoring	LSTM	Tuple of Monitoring Data	Monitoring Thresholds	Generated by themselves	✓	×
Ujjan et al [23]	Benign or Malicious traffic	Snort Stacked Autoencoders	sFlow	Benign or Malicious	Generated by themselves	✓	×
Iliadis and Kaifas [17]	Darknet traffic	KNN, MLP, RF, DT and GB	Flow Statistics Tuple	Darknet or non-Darknet	ISCXVPN2016 ISCXTor2017	×	×
Jadav et al. [18]	Darknet traffic	CNN Supervised classifiers	Flow Statistics Tuple	Darknet or non-Darknet	ISCXVPN2016 ISCXTor2017	×	×
Marim et al. [15]	Darknet traffic	DT and RF	Statistics Tuple	Darknet or non-Darknet	ISCXVPN2016 ISCXTor2017	×	×
Singh et al. [3]	Darknet traffic	CNN Supervised classifiers	Image	Darknet or non-Darknet	ISCXVPN2016 ISCXTor2017	×	×
Our approach	**Darknet traffic**	**CNN combined with RL**	**Image**	**Darknet or non-Darknet**	**ISCXVPN2016 ISCXTor2017**	✓	✓

3. Proposed method

To deal with the online classification of Darknet traffic, we have proposed a monitoring method[1] that combines AI techniques in order to predict Darknet and non-Darknet traffic in real time, using RL to sample the network in a intelligent manner. In our approach, we combine CNNs with RL techniques to carry out online sampling and prediction of Darknet packets, as shown in Fig. 2. Our RL agent is based on a Deep Q-Network (DQN), in which Q-Learning is combined with deep learning. The lower part refers to the RL agent's operating environment. Our packet sampling proposal for Darknet traffic prediction is attached as a daemon to the interfaces of both network entities and virtualized services to monitor the network traffic in an intelligent manner.

From right to left, Fig. 2 shows an *action*, which comes from the RL algorithm, and the Adaptive Sampling agent, which collects samples from the network based on the number of packets suggested by the *action*. At the end of the packet sampling stage, the packets are submitted to a CNN to predict the type of traffic class. In previous work, we proposed and evaluated the Packet Vision [25] method, which classifies network packets into classes of applications using CNNs. In this paper, we present a novel online classification method for TOR and *non*-TOR traffic.

Fig. 2 exemplifies two universes, consisting of the environment and the agent. The environment is where actions are applied, whereas the agent is responsible for applying the action within the environment and subsequently evaluating its effectiveness. The environment contains the monitored entities/interfaces in our traffic classification scenario, while the agent is the entity that monitors the entities/interfaces for malicious traffic. In the malicious traffic prediction process considered here, where an Adaptive Sampling Agent performs within an environment, our RL algorithm stores the information about the environment, the action taken and the state in the Replay Memory $\mathcal{D}$. Thus, $\mathcal{D}$ supports the learning process where a fixed set of previously taken batch actions are randomly taken to feed the learning algorithm.

At the end of the packet sampling classification stage, our method considers the percentage of the TOR class and allocates a reward when it is above the objective threshold of the RL algorithm. After operating within the environment, the RL stores the tuple containing the state, action, reward, and next state in $\mathcal{D}$. In this paper, we use a *Q-Learning* algorithm that feeds data stored in

[1] Source code available on https://github.com/romoreira/adaptive-monitoring.

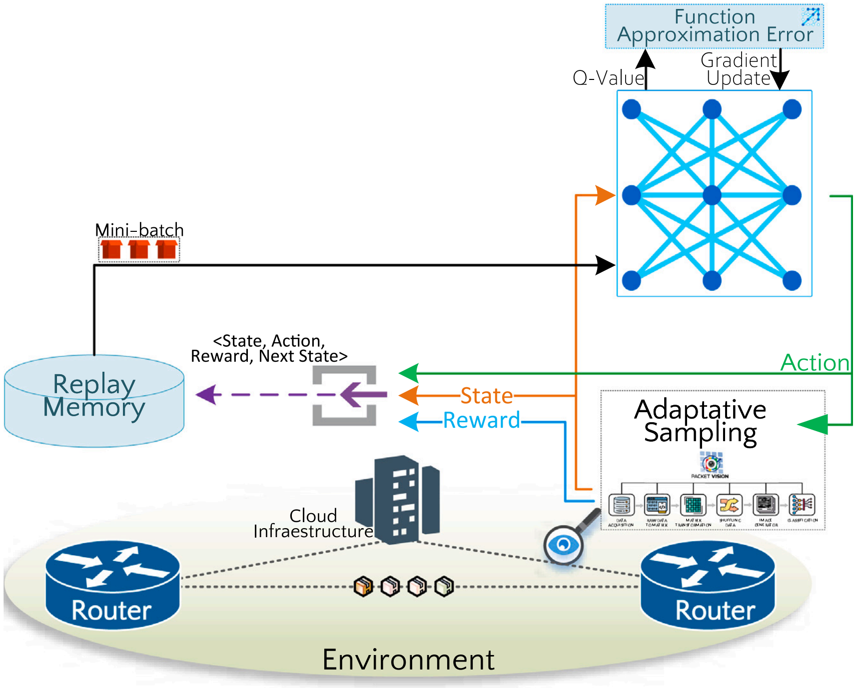


Fig. 2. Adaptive sampling combined with RL.

$\mathcal{D}$ for its training. An approximation function is applied to the predicted Q-Value, and the network weights are updated throughout its execution.

Our proposed method is based on Algorithm 1, which estimates the number (rate) of packets that the Sampling Agent should gather. This number should be as close as possible to the target percentage of TOR traffic. *Q-Learning* is a policy of RL that aims to maximize the total return. A *Q-Table* is used to store the $< state, action >$ tuple that represents the state of the environment and the action that the agent should take.

Algorithm 1: *Q-Learning* Algorithm for Adaptive Network Sampling.

Input: Q^* Network, Weights Θ
Data: TOR Traffic Percent (%)
Init: $\mathcal{D}$ with capacity N, state s_t

```
for episode = 1, M do
    for t = 1, T do
        With probability ϵ select a random action a_t
        Otherwise select a_t = max_a Q*(s_t,a;Θ)
        Sample Network according to action a_t and observe reward r_t and state s_{t+1}
        Store Transition (s_t, a_t, r_t, s_{t+1}) in D
        Set s_{t+1} = s_t
        Sample random minibatch of transitions (s_t, a_t, r_t, s_{t+1}) from D
        The agent sets according to Bellman Equation: Q*(s_t,a_t) ← Q(s_t,a_t) + α (learning rate) × ( r_t (reward) + γ (discount factor) × maxQ(s_{t+1},a) − Q(s_t,a_t)), In cases where
          episode terminates at t, Q*(s_t,a_t) = R(s_t,a_t)
        Update the Weights Θ for the gradient-based evaluation
        Episode index is updated t ← t + 1
```

Bellman update (line 9): $Q^*(s_t,a_t) \leftarrow Q(s_t,a_t) + \underbrace{\alpha}_{\text{learning rate}} \times (\underbrace{r_t}_{\text{reward}} + \underbrace{\gamma}_{\text{discount factor}} \times maxQ(s_{t+1},a) - Q(s_t,a_t))$, In cases where episode terminates at t, $Q^*(s_t,a_t) = R(s_t,a_t)$

The *Q-Learning* algorithm has two ways of interacting with the environment, which are know as exploiting and exploring. Exploiting refers to the choice of action, based on the maximum reward that will be gained from this action, whereas the aim of exploring is to randomly choose a environment according to a ϵ in order to look for new states that may not be chosen in the exploiting phase.

In our method, Algorithm 1 used to combine RL with a CNN to carry out traffic prediction based on Packet Vision, in order to classify TOR and *non*-TOR traffic. In the initial phase of Algorithm 1, a Q-Network Q^* and its weights Θ are given as input. The data input to and generated in the algorithm refer to the calculated percentage of TOR traffic. At the initialization stage, the algorithm assigns $\mathcal{D}$ the maximum capacity N, and the state tuple s_t is randomly initialized.

Algorithm 1 contains two nested loops, which carry out exploration and exploitation of the environment. The outermost loop represents the exploration of the environment in each episode, ranging from 1 to $\mathcal{M}$ while the innermost loop represents the environmental exploitation of each episode; that is, within a single iteration, it is possible to explore the environment according to a T limit.

In line 3 of Algorithm 1, an action a_t is taken based on a probability ϵ. We describe this action in more detail in Algorithm 2, which determines whether exploitation or exploration will occur using an Epsilon-Greedy policy. Algorithm 2 chooses a random action within the Action Space, from the range 1 to $\mathcal{N}$, where $\mathcal{N}$ is the maximum number of sampled packets, or chooses the action with the highest estimated reward most of the time.

In our approach, the Action Space is Discrete, which allows us to vary the number of packets sampled from zero (0) to 2000. The algorithm relies on a number of sampling packets collected from the network interface to measure the percentage of Darknet traffic running on top of it. The Reward function is based on the highest percentage of Darknet traffic after a given sampling of the network interface.

Algorithm 2: Epsilon-Greedy Action Selection.

```
Data: Q-Table, S, state
Output: Selected Action
Function SelectAction(Q-Table, S, ε):
    n ← uniform random number (0, 1)
    if n < ε then
        A ← random action from the Action Space
    else
        A ← maxQ(S,.)
                                    // which is the action with the highest estimated reward
    return A
```

On line 5 of Algorithm 1, where most of the novelty of our work resides, the agent interacts with the environment to sample and classify online packets looking for Darknet traffic. The action taken in this environment requires to store the tuple of $< state, action >$ and updating of the new state, as shown in lines 6 and 7. In line 8, the algorithm takes a sample of transitions from $\mathcal{D}$, and passes them as input to the DQN for *Q-value* prediction. In line 9, the agent chooses an action and observes its reward, calculated according to the Bellman equation, before updating the network values. In line 10, the Θ weights of the *Q-Learning* algorithm are updated, and the exploration of the environment is continued in line 11.

4. Experimental testbed

To functionally validate our proposal, we carried out experiments according to Fig. 3. On a machine Intel(R) Core(TM) i7-7500U CPU @ 2.70 GHz 2.90 GHz with 8 GB RAM, NVIDIA GeForce 940MX 2 GB Video Card and Ubuntu 18.04 LTS, we configure virtual interfaces named *veth0* and *veth1*.

As the bottom part of Fig. 3, packets flow in a loop from left (*veth0*) to the right through the *tcpreplay*[2] tool. We configure the transmission rate, packet interval, and the interval between the complete transmission of files randomly. When packets arrive at the right interface (*veth1*), they are captured by the Sampling Agent that submits one to Packet Vision to make them able to be predicted by a CNN previously trained for that dataset.

In the upper part of Fig. 3, the packets of a given sample are classified as TOR and *non*-TOR. The RL algorithm feeds the percentage of packets within the number of packets per period or by dataset size.

The experimental testbed comprises the dataset used to train the CNN. For *non*-TOR traffic, we consider the ISCXVPN2016 [26], and for TOR traffic, we consider the ISCXTor2017 [24] datasets. For a fair and better comparison with the literature, we experimented with the same dataset used by other works presented in Table 1. Each dataset contains eight (8) traffic categories: browsing, email, chat, audio-streaming, video-streaming, File Transfer Protocol (FTP), Voice over IP (VoIP), Peer-to-Peer (P2P). As Fig. 4, each service type was categorized according to the traffic origin. The size of the datasets used to evaluate the system is 3196 images *non*-TOR and 2892 images TOR. We split the dataset as training/validation/test (80/10/10).

We guarantee that the dataset contained packets of both TOR and *non*-TOR classes in a balanced way. In addition, we ensure that the merge of the packets of each class in the *torNonTor.pcap* was distributed randomly. To ensure class balancing, we create a new *non*-TOR traffic dataset based on the previous one [24,26] containing the traffic classes: Audio-Stream, Browsing, Chat, Email, P2P, FTP, Video-Stream, and VoIP as a single *non*-TOR class label, leading to distribution as in Fig. 4.

We tested three CNNs architectures: ResNet, SqueezeNet, and DenseNet pre-trained on the publicly available ImageNet dataset. We chose these CNNs because of their good performance in previous network traffic classification work [25].

- **ResNet:** is a deep residual network based on the concept of residual learning. This CNN mainly overcomes the overfitting and degradation problem by introducing residual connections. We evaluated ResNet with 34 layers: one standalone convolution layer and 16 residual bocks followed by one fully connected layer [27].

[2] Available in https://tcpreplay.appneta.com/.

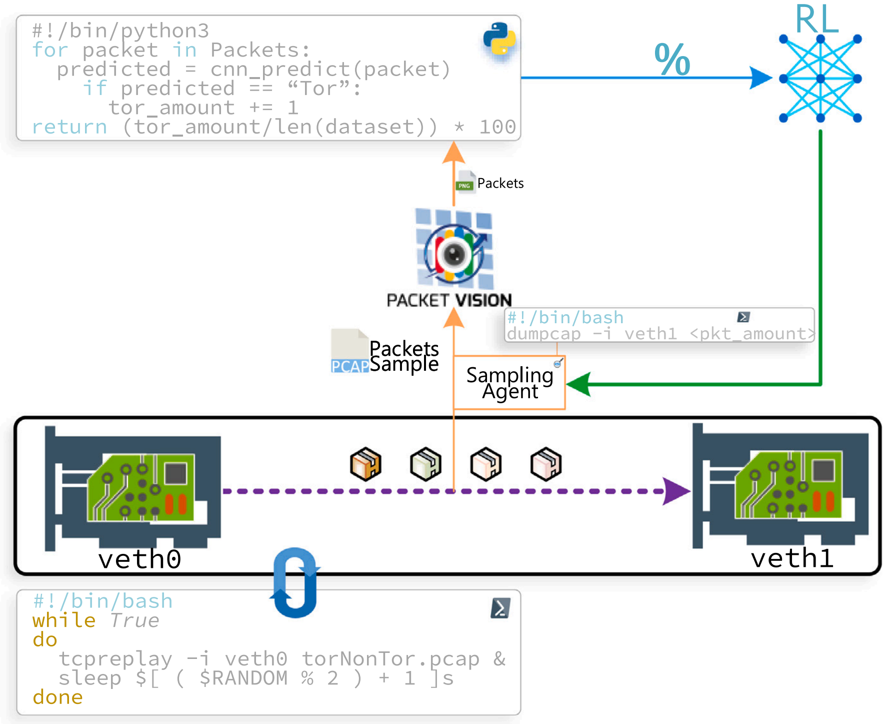


Fig. 3. Experimental setup of adaptive sampling for TOR traffic prediction.

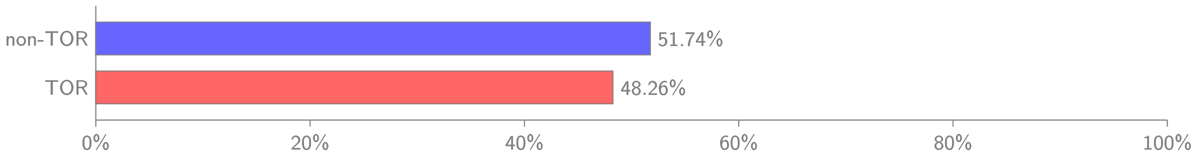


Fig. 4. Distribution of dataset classes.

Table 2
Experimental setup parameters.

Strategy	Parameters	Values
DQN	Exploration/Exploitation factor (ϵ)	0.01
	Batch size	2
	Discount factor (γ)	0.99
	Learning rate (α)	0.9
	Reward (r)	2
	Episodes	200
CNN	Batch size	32
	Epochs	50
	Optimizer	Stochastic Gradient Descent (SGD)
	Learning rate	0.001
	Momentum	0.9

- **SequeezeNet:** comprises convolutional layers, pooling layers, and fire layers. This CNN does not have fully connected layers, but the fire layers perform the same functions of a fully connected layer. The main advantage is that it performs analyses successfully by reducing the number of parameters, thereby decreasing the model size capacity [28].
- **DenseNet:** uses dense blocks to concatenate the input feature maps of each former sub-block and so use them as the input feature map of a certain sub-block. This CNN uses dense connectivity to overcome vanishing gradient problems and reduce the number of parameters [29]. In this study, we use the DenseNet with 169 layers.

For the sake of reproducibility, we considered the parameters defined in Table 2.

We trained the CNNs using SGD optimization algorithm, applied to minimize the cross-entropy loss function. This function is responsible for quantifying the loss by accepting the prediction generated by the current parameters of the model. As defined in Eq. (1), given a current set of parameters W, the total cost based on a training subset containing N instances is usually computed by averaging the costs of all X_i samples, and the known classes y_i, where i represents the classes TOR and *non*-TOR.

$$\mathscr{L}(W) = \frac{1}{n}\sum_{i=1}^{N}(y_i, f(x_i; W)) \tag{1}$$

Furthermore, the classification performance was calculated considering the indices obtained from the confusion matrix. These indices were used to compute the accuracy metric, which is the hits of the classifier as a whole (Eq. (2)).

$$Accuracy = \frac{T_P + T_N}{T_P + T_N + F_P + F_N} \tag{2}$$

where values:

- True Positive (T_P): refers to the positive predicted samples that were correctly labeled by the classifier.
- True Negative (T_N): refers to the negative predicted samples that were correctly labeled by the classifier.
- False Positive (F_P): refers to the positive predicted samples that were incorrectly labeled as positive.
- False Negative (F_N): refers to the positive predicted samples that were mislabeled as negative.

After building a dataset (*torNonTor.pcap*), we carried out experiments with the aim of answering the following Research Questions (RQs):

RQ1: Of the CNNs considered for the experimental testbed, which perform better in terms of accuracy in predicting Darknet and non-Darknet traffic?
RQ2: Which CNN takes the shortest time to predict the traffic class of a network packet (Darknet and non-Darknet)?
RQ3: What is the behavior of the RL algorithm in terms of intelligent network packet sampling?
RQ4: How can network monitoring leverage RL for online traffic prediction?

5. Results and discussion

We evaluated the proposed methodincluding each of its components and the interactions between them. The first experiment aimed to investigate which CNNs performed better in terms of prediction accuracy, to answer the RQ1 and to find the best CNN model for embedding TOR and *non*-TOR traffic prediction into a sampling agent. We found that the overall performance of the model was 99.84% for both DenseNet and ResNet, while SqueezeNet reached an accuracy of 99.35%. The learning behavior of the CNNs in the TOR and *non*-TOR traffic prediction task, as shown in Fig. 5, suggests a good level of generalization and correctness of the model after training.

All of the CNNs performed well on the traffic prediction task, leading to the need for a deeper evaluation before incorporating one into an online network traffic classification scenario. To do this, we carried out experiments to explore the time spent by each CNN on the prediction task. It can be seen from Fig. 6 that the CNN required the least time (in seconds s) to classify a single network packet as TOR or *non*-TOR traffic was SqueezeNet.

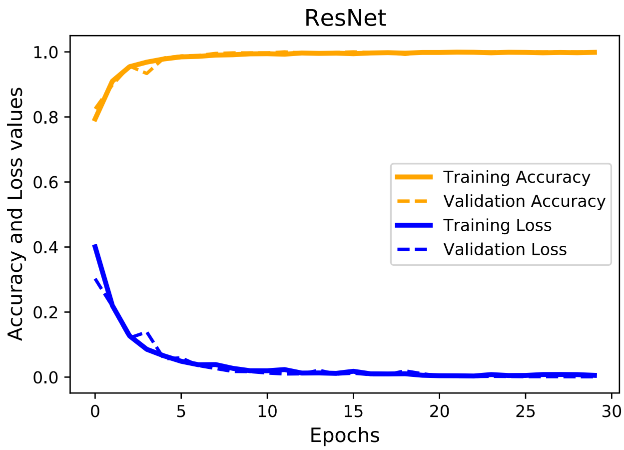
From the point of view of accuracy alone, SqueezeNet would be the least suitable choice as it has a lower prediction accuracy than the alternatives. However, an online prediction scenario is more sensitive to performance than accuracy. Thus, the percentage increase in accuracy achieved by choosing a CNN that classify spending more time is ≈0.59%. While the time spent percentage decrease saves in packet prediction task choosing a CNN with an accuracy sightly minor (≈0.59%) is 51.85%, leading us to choose SqueezeNet for the online classification task.

Making suggestive admit that the prediction time for the TOR and *non*-TOR classification scenario picks faster CNNs than those that are more accurate. Hence, SqueezeNet, despite having a slightly lower accuracy, was superior to the others in terms of packet prediction time for the classification of TOR and *non*-TOR traffic, spending an average of ≈0.027 s. This prediction time can be further accelerated using a suitable hardware platform such as Field Programmable Gate Array (FPGA). In answer to RQ2, we see that SqueezeNet requires the shortest time for the task of packet classification in the online ranking scenario.

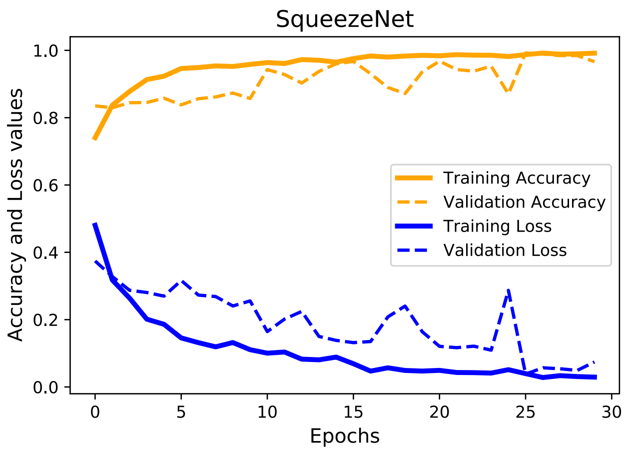
Additional experiments were conducted to investigate how the *Q-Learning* algorithm handled network sampling for online traffic classification. We compared a random sampling approach with the proposed method in Algorithm 1. Our experimental testbed contained 48.26% of TOR traffic (see Fig. 4), which was randomly mixed with *non*-TOR packets. With random sampling of the network, the RL algorithm seemed to be unable to pursue a target of TOR traffic percentage as shown in Fig. 7.

In high-performance networks, monitoring mechanisms should not impose an overhead on the monitored entities. As predicted by the SFLow Tool, the measurement accuracy does not depend on the number of total captured frames in a network but on the number of samples. The proposed monitoring method for classification therefore combines CNNs with RL to predict TOR and *non*-TOR traffic on a high-performance network where the number of packets (observations) is not fixed.

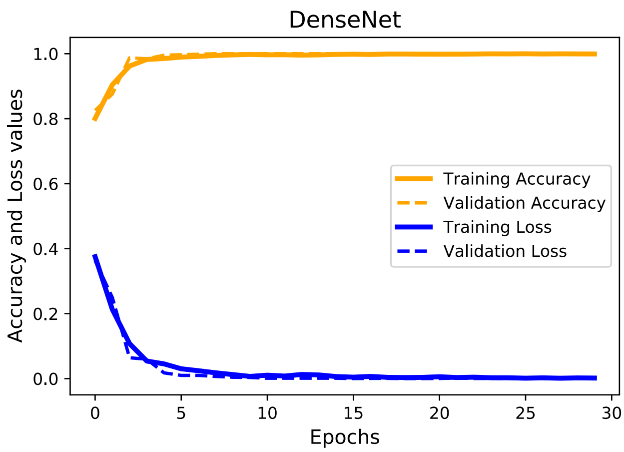
The behavior of the loss values and rewards in the random sampling strategy was unsatisfactory. The loss refers to the difference between the predicted value and the target and is used to assess how poorly or well the model behaves after each optimization iteration. In this study, we used the MSE function to calculatethe *loss* over the 200 experimental episodes. Fig. 7 shows that using



(a)



(b)



(c)

Fig. 5. Learning behavior in training phase considering training and validation sets.

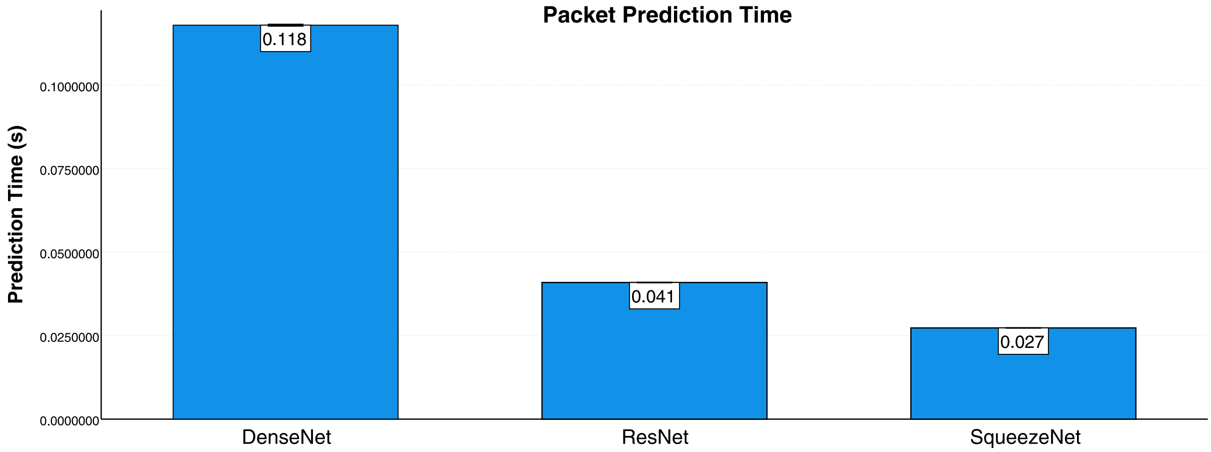


Fig. 6. Intelligent network sampling: prediction time for each CNN.

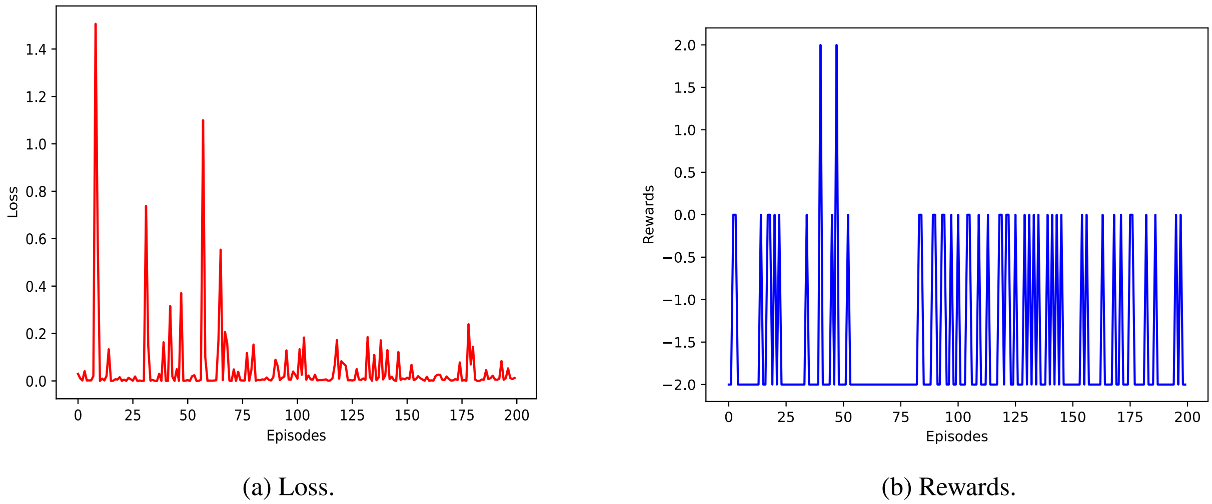


Fig. 7. Learning behavior in random sampling.

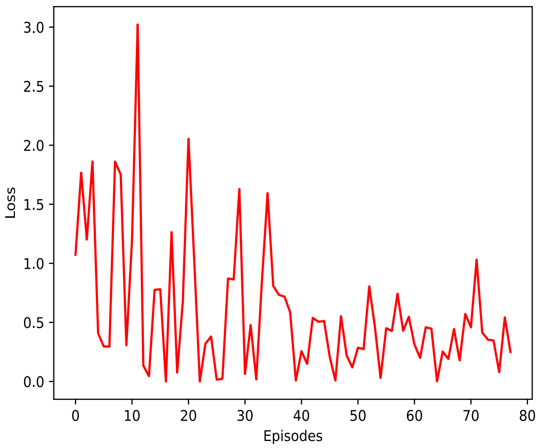
a random approach not produce reward accumulation, the model did not converge to a satisfactory result, and atypical learning behavior was shown throughout the episodes.

In contrast, Fig. 8 shows that the use of RL for specification of the online packet sampling rate appears to be suitable for the proposed scenario. It is possible to perceive the property of generalization and stabilization of rewards achieved after the algorithm pursues the TOR traffic rate with a previously defined percentage of TOR traffic (48.26%) over ≈80 episodes. Fig. 8 shows the progressive accumulation of rewards and learning over episodes, where both TOR and *non*-TOR traffic predictions occur and answer RQ3.

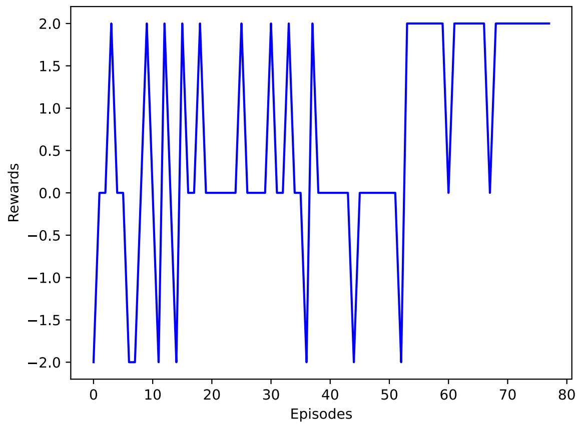
In addition to our scenario involving TOR/*non*-TOR traffic prediction, real applications and other monitoring platforms can use replay memory which contains the last successful actions of the $Q - Learning$ algorithm. Thus, in answer to RQ4, we see that other monitoring techniques for high-performance networks can be combined to carry out prediction in other contexts and other classes of applications, leading us to conclude that RL is an excellent technological enabler for online traffic prediction.

In order understand which features of the TOR and *non*-TOR packets provided most activation for the CNNs we applied an Ablation-based Class Activation Mapping (Ablation-CAM) [30] to view the activated patterns. We chose this approach because it can handle the gradient saturation that occurs in traditional Grad-CAM and creates uninformative regions in the image. We observed that the DenseNet and ResNet models identified better texture patterns and thus achieved better classification performance. Fig. 9 shows the visualizations generated by Ablation-CAM, in which the red and yellow areas in the heat map represent the regions with the highest activation and the blue and green areas represent regions with less activation. From this sample, we can infer that there is a structural difference in the packets that occasionally occurs because the TOR packets are encrypted, unlike the *non*-TOR packets.

Finally, we compared the best result achieved in this paper with other state-of-art approaches in the literature. The best result in our study was obtained with the DenseNet architecture, which gave an accuracy of 99.84%. The results reported in the literature



(a) Loss.



(b) Rewards.

Fig. 8. Learning behavior in adaptive sampling.

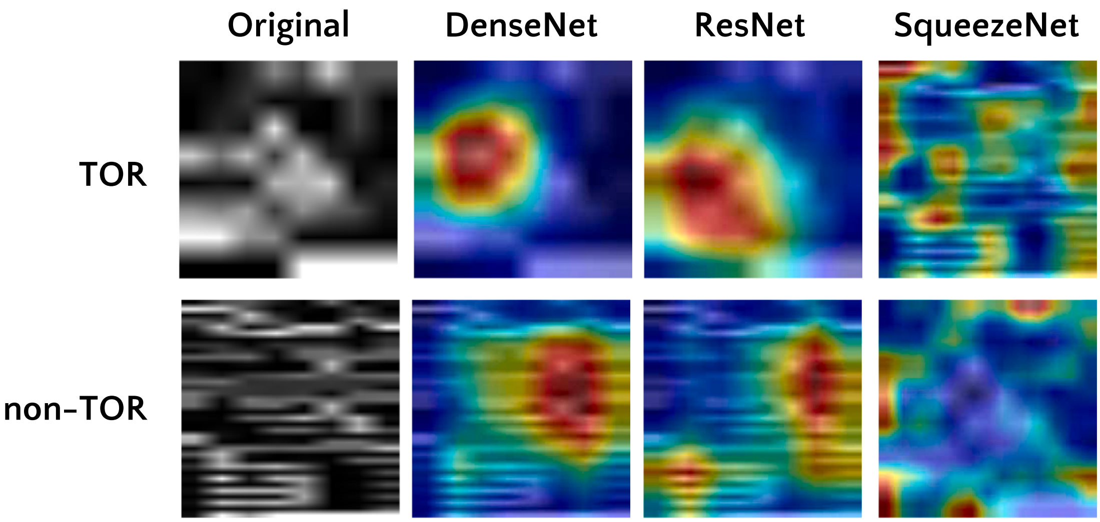


Fig. 9. Visualization using Ablation-CAM on the TOR (top) and *non*-TOR (bottom) classes showing activation maps for each CNN evaluated.

for the ISCXVPN2016 and ISCXTor2017 datasets are presented in Table 3, and it can be seen that our best score is higher than that of the best state-of-the-art technique reported in the literature. Furthermore, our method allows for near to real-time classification and intelligent sampling of packets, and our traffic classification strategy, in which the entire packet structure (header and payload) is considered, makes our method invariant to modifications in the of profile of the malicious traffic.

For a fair comparison, Table 3 shows approaches that used the same datasets as us, despite using different learning approaches [24,26]. Our approach differs from the scheme of Singh et al. [3], which also used binary classification, as it groups all *non*-TOR traffic classes from the dataset (audio-stream, browsing, chat, email, P2P, transfer, video-stream, and VoIP) under a single *non*-TOR class label, to validate our near-to-real-time traffic classification and our intelligent sampling based on RL.

The rationale for this grouping was to enable efficient execution in the experimental scenario, in which we replayed two synthetic data streams (TOR and *non*-TOR) and transmitted them between two interfaces in order to investigate the optimal packet sampling rate for Darknet classification using RL, requiring packets amount per class equivalent. To avoid disparity in the number of instances per class, it was necessary to resize the original *non*-TOR, leading to training, validation, and testing with fewer data than the original dataset. Our approach was found to be suitable for Darknet/*non*-Darknet classification in high-performance networks, due to our intelligent and optimal packet sampling method, and open up avenues for further investigations of real-time traffic classification.

Table 3
Comparison with literature.

Approach	Method	Real-time classification	Intelligent sampling	Accuracy (%)
Lashkari et al. (2020) [16]	CNN	×	×	86
Iliadis and Kaifas (2021) [17]	KNN, MLP, RF, DT, and GB	×	×	98
Jadav et al. (2021) [18]	CNN and supervised classifiers	×	×	99
Marim et al. (2021) [15]	DT and RF	×	×	99
Singh et al. (2021) [3]	CNN and supervised classifiers	×	×	94.89
Our approach	**CNN combined with RL**	✓	✓	**99.84**

6. Concluding remarks

This paper has proposed an intelligent method for the online monitoring and classification of TOR and *non*-TOR traffic. Our work advances the state of the art by combining CNNs with RL to estimate, near real-time, the best network sampling rate that allows the algorithm to reach an estimate of the TOR traffic running through the network.

Our analysis revealed that many network traffic classification methods are needed to handle the online traffic classification of Darknet properly. Furthermore, these methods realize packet sampling with fixed and pre-defined parameters, disregarding the seasonal traffic volumes, and possibly leading to inaccurate estimates of malicious traffic.

Of the main contributions made by this work, we note in particular that CNNs were shown to be suitable for predicting TOR and *non*-TOR traffic, and reached accuracies that were compatible to state-of-the-art alternatives, but going further with an adaptive and online prediction method. In addition, we evaluated the suitability of using the Graphics Processing Unit (GPU) packet classification.

In this study we discovered a new method for better adaptive sampling of packets for online traffic prediction. Our approach opens up new opportunities in this field and offers a method for the governments to apply national crackdowns and supervision to avoid crimes, thus making the Internet and other online activities connectivity safer.

This study suggests new avenues for the investigation of new lightweight monitoring and classification methods for classifying malicious traffic that consider the seasonality of the network. In future, we intend to evaluate other *Q-Learning* algorithms, and other reward policies; to apply our method to other datasets with different attributes such as size, distribution, and dimensionality; to consider other monitorable characteristics of the entities in order to construct other datasets; and, finally, to use sophisticated traffic monitoring methods to support cyber security efforts.

CRediT authorship contribution statement

Rodrigo Moreira: Conceptualization, Methodology, Software, Investigation, Visualization, Writing – original draft, Review & editing. **Larissa Ferreira Rodrigues Moreira:** Software, Methodology, Visualization, Writing – original draft, Review & editing. **Flávio de Oliveira Silva:** Funding acquisition, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

We acknowledge the financial support of the Brazilian National Council for Scientific and Technological Development (CNPq), grant #421944/2021-8. Also, this study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil (CAPES) – Finance Code 001 and the National Education and Research Network (RNP) for financial support under the CT-Mon call.

References

[1] Tsai P-W, Tsai C-W, Hsu C-W, Yang C-S. Network monitoring in software-defined networking: A review. IEEE Syst J 2018;12(4):3958–69.
[2] Aceto G, Botta A, Marchetta P, Persico V, Pescapé A. A comprehensive survey on internet outages. J Netw Comput Appl 2018;113:36–63.
[3] Singh D, Shukla A, Sajwan M. Deep transfer learning framework for the identification of malicious activities to combat cyberattack. Future Gener Comput Syst 2021;125:687–97.
[4] Zhao F, Zhou J, Nie C, Huang H, Jin H. SmartCrawler: A two-stage crawler for efficiently harvesting deep-web interfaces. IEEE Trans Serv Comput 2016;9(4):608–20.
[5] Sarwar MB, Hanif MK, Talib R, Younas M, Sarwar MU. DarkDetect: Darknet traffic detection and categorization using modified convolution-long short-term memory. IEEE Access 2021;9:113705–13.

[6] Bazli B, Wilson M, Hurst W. The dark side of I2P, a forensic analysis case study. Syst Sci Control Eng 2017;5(1):278–86.
[7] Xu J, Ju A. Detect Darknet URL based on artificial neural network. In: The 5th international conference on computer science and application engineering. New York, NY, USA: Association for Computing Machinery; 2021, p. 1–6, URL: https://doi.org/10.1145/3487075.3487132.
[8] Montieri A, Ciuonzo D, Bovenzi G, Persico V, Pescapé A. A dive into the dark web: Hierarchical traffic classification of anonymity tools. IEEE Trans Netw Sci Eng 2020;7(3):1043–54.
[9] Rodrigues LF, Backes AR, Travençolo BAN, de Oliveira GMB. Optimizing a deep residual neural network with genetic algorithm for acute lymphoblastic leukemia classification. J Digit Imaging 2022.
[10] Ren X, Gu H, Wei W. Tree-RNN: Tree structural recurrent neural network for network traffic classification. Expert Syst Appl 2021;167:114363.
[11] Lotfollahi M, Jafari Siavoshani M, Shirali Hossein Zade R, Saberian M. Deep packet: a novel approach for encrypted traffic classification using deep learning. Soft Comput 2020;24(3):1999–2012.
[12] Shafiq M, Tian Z, Bashir AK, Jolfaei A, Yu X. Data mining and machine learning methods for sustainable smart cities traffic classification: A survey. Sustainable Cities Soc 2020;60:102177.
[13] Nguyen TG, Phan TV, Hoang DT, Nguyen TN, So-In C. Federated deep reinforcement learning for traffic monitoring in SDN-based IoT networks. IEEE Trans Cogn Commun Netw 2021;7(4):1048–65.
[14] Dias KL, Pongelupe MA, Caminhas WM, de Errico L. An innovative approach for real-time network traffic classification. Comput Netw 2019;158:143–57.
[15] Marim M, Ramos P, Oliveira R, Vieira A, Silva E. Caracterização e classificação do tráfego da Darknet com modelos baseados em árvores de decisão. In: Anais do XXXIX simpósio brasileiro de redes de computadores e sistemas distribuídos. Porto Alegre, RS, Brasil: SBC; 2021, p. 127–40, URL: https://sol.sbc.org.br/index.php/sbrc/article/view/16716.
[16] Habibi Lashkari A, Kaur G, Rahali A. DIDarknet: A contemporary approach to detect and characterize the Darknet traffic using deep image learning. In: 2020 the 10th international conference on communication and network security. ICCNS 2020, New York, NY, USA: ACM, Association for Computing Machinery; 2020, p. 1–13. http://dx.doi.org/10.1145/3442520.3442521.
[17] Iliadis LA, Kaifas T. Darknet traffic classification using machine learning techniques. In: 2021 10th international conference on modern circuits and systems technologies. MOCAST, IEEE; 2021, p. 1–4. http://dx.doi.org/10.1109/MOCAST52088.2021.9493386.
[18] Jadav N, Dutta N, Sarma HKD, Pricop E, Tanwar S. A machine learning approach to classify network traffic. In: 2021 13th international conference on electronics, computers and artificial intelligence. ECAI, IEEE; 2021, p. 1–6. http://dx.doi.org/10.1109/ECAI52376.2021.9515039.
[19] Velan P, Čermák M, Čeleda P, Drašar M. A survey of methods for encrypted traffic classification and analysis. Int J Netw Manage 2015;25(5):355–74.
[20] Aceto G, Ciuonzo D, Montieri A, Pescapè A. MIMETIC: Mobile encrypted traffic classification using multimodal deep learning. Comput Netw 2019;165:106944.
[21] Zheng W, Gou C, Yan L, Mo S. Learning to classify: A flow-based relation network for encrypted traffic classification. In: Proceedings of the web conference 2020. New York, NY, USA: Association for Computing Machinery; 2020, p. 13–22, URL: https://doi.org/10.1145/3366423.3380090.
[22] Mijumbi R, Asthana A, Koivunen M, Haiyong F, Zhu Q. Design, implementation, and evaluation of learning algorithms for dynamic real-time network monitoring. Int J Netw Manage 2021;31(4):e2108, e2108 nem.2108.
[23] Ujjan RMA, Pervez Z, Dahal K, Bashir AK, Mumtaz R, González J. Towards sFlow and adaptive polling sampling for deep learning based DDoS detection in SDN. Future Gener Comput Syst 2020;111:763–79.
[24] Lashkari AH, Gil GD, Mamun MSI, Ghorbani AA. Characterization of tor traffic using time based features. In: Proceedings of the 3rd international conference on information systems security and privacy - Volume 1: ICISSP. INSTICC, SciTePress; 2017, p. 253–62. http://dx.doi.org/10.5220/0006105602530262.
[25] Moreira R, Rodrigues LF, Rosa PF, Aguiar RL, Silva FdO. Packet Vision: a convolutional neural network approach for network traffic classification. In: 2020 33rd SIBGRAPI conference on graphics, patterns and images. SIBGRAPI, IEEE; 2020, p. 256–63. http://dx.doi.org/10.1109/SIBGRAPI51738.2020.00042.
[26] Draper-Gil G, Lashkari AH, Mamun MSI, Ghorbani AA. Characterization of encrypted and VPN traffic using time-related features. In: Proceedings of the 2nd international conference on information systems security and privacy - Volume 1: ICISSP. INSTICC, SciTePress; 2016, p. 407–14. http://dx.doi.org/10.5220/0005740704070414.
[27] He K, Zhang X, Ren S, Sun J. Deep residual learning for image recognition. In: Proceedings of the IEEE conference on computer vision and pattern recognition. CVPR, IEEE; 2016, p. 770–8.
[28] Iandola FN, Moskewicz MW, Ashraf K, Han S, Dally WJ, Keutzer K. SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <1MB model size. 2016, CoRR abs/1602.07360.
[29] Huang G, Liu Z, Van Der Maaten L, Weinberger KQ. Densely connected convolutional networks. In: Proceedings of the IEEE conference on computer vision and pattern recognition. IEEE; 2017, p. 4700–8.
[30] Fu R, Hu Q, Dong X, Guo Y, Gao Y, Li B. Axiom-based Grad-CAM: Towards accurate visualization and explanation of CNNs. 2020, CoRR abs/2008.02312.

Rodrigo Moreira is a Professor at the Federal University of Viçosa and received his B.S. degree from the Federal University of Viçosa and his M.S. degree from the Federal University of Uberlândia, Brazil, in 2014 and 2017 respectively. Also, he received in 2021 a Ph.D. degree in Computer Science at the Federal University of Uberlândia. His research interests include future internet, quality of service, cloud computing, network function virtualization, software-defined networking, and edge computing.

Larissa Ferreira Rodrigues Moreira is a Ph.D. candidate at the Federal University of Uberlândia, Brazil. She received the B.Sc. degree in Computer Information Systems (2016) and the M.Sc. degree in Computer Science (2018) from the Federal University of Viçosa, Brazil. Larissa is a Professor at the Federal University of Viçosa. Her research interests include Image Processing, Computer Vision and Machine Learning.

Flávio de Oliveira Silva is a Professor at the Faculty of Computing (FACOM) in the Federal University of Uberlândia (UFU) and received a Ph.D. degree in 2013 from the University of São Paulo. Member of ACM, IEEE, and SBC, he has several papers published and presented in conferences around the world. He is a reviewer for several journals and member of TCPs of several IEEE conferences. Future Networks, IoT, Network Softwarization (SDN and NFV), Future Intelligent Applications and Systems, Cloud Computing, and, Software based Innovation are among his main current research interests.